

UAA4

Projet collaboratif

Nom/Prénom :

Classe :

		Pondération	Pondération finale
Q1	Implémentation du projet	/100	/45
Q2	Dossier de conception	/100	/25
Q3	Défense Orale	/100	/30
TOTAL /100			

1. Description générale

Vous aurez pour objectif de créer un jeu interactif

- au choix parmi une liste prédéfinie (vous trouverez le détail de cette liste dans les annexes).
- ou bien que vous proposerez vous-même. Attention, dès lors les règles devront être **STRICTEMENT** définies et validées par le professeur.

Le projet sera approché comme un réel projet informatique dans le sens où il suivra la différentes phases de conception*.

À savoir :

- Définition du projet et définition des ressources.
- Analyse et conception.
- Implémentation du projet.
- Phase de test et déploiement

** Au complet, les phases de conceptions d'un projet informatique sont étendues au nombre de 7.*

2. Planification du projet sur l'année et livrables

La planification se fait via Classroom. L'élève est tenu de consulter régulièrement cette plateforme afin de prendre connaissance des délais de remise des travaux tout au long de l'année.

Chaque groupe doit créer un tableau partagé avec le professeur afin de piloter le projet en temps réel.

- Décomposition en tâches : le projet doit être découpé en cartes précises (ex: « Créer la fonction de déplacement », « Réaliser le menu principal »).
- Attribution et échéances : chaque carte doit être assignée à un membre du groupe avec une date de fin estimée.
- État d'avancement (Kanban) : le tableau doit refléter la progression via trois colonnes :
 - À faire (To Do) : Tâches planifiées.
 - En cours (Doing) : Travail actuel. Une carte ici signifie que l'élève peut expliquer sa logique au professeur à tout moment.
 - Terminé (Done) : Tâches finalisées et testées.
- Carnet de bord technique : À chaque remise, le groupe fait le point sur la charge de travail restante via Trello. Les élèves utiliseront les commentaires des cartes pour lister les points inconnus (difficultés techniques) et décrire la manière dont ils comptent les résoudre.

Outil obligatoire : <https://trello.com>

3. Dossier de conception et d'analyse du projet

Ce dossier reprendra :

1. La définition du projet
2. Le descriptif de l'interface du logiciel et l'entièreté des cas d'utilisation
3. Les tâches à réaliser
4. Des aperçus visuels
5. Rétrospectives
6. Perspectives

De plus, la première page du dossier comportera une page de garde reprenant :

- le logo de l'école

- l'année scolaire
- les noms et prénoms des membres du groupe
- une image illustrative

3.1. Définition du projet

Cette section reprend la définition complète du projet. En d'autres termes :

- Quel sera le jeu implémenté ?
- Quelles seront les règles du jeu ?
- Comment y jouer ?

3.2. Descriptif de l'interface et cas d'utilisation

Cette section décrira l'ensemble des écrans proposés par le logiciel. En vis à vis de ces écrans, vous détaillerez tous les cas d'utilisation possibles pour cette partie de l'application.

Outil recommandé : <https://www.draw.io>

3.3. Tâches à réaliser

Chaque groupe doit identifier et documenter les tâches nécessaires à la réalisation complète du jeu.

Le projet doit être découpé en tâches précises (en lien avec la planification du projet).

Pour chaque tâche, le dossier de conception doit obligatoirement préciser :

- Un titre clair.
- Une brève description de ce qui doit être réalisé.
- Le ou les membre(s) responsable(s) de la tâche (si la tâche est réalisée en duo, les deux noms doivent apparaître).

La planification (dates), l'état d'avancement (colonnes Kanban) et sont gérés exclusivement via Trello et ne doivent pas apparaître dans le dossier de conception.

3.4. Aperçus visuels

Cette section présentera un ou plusieurs visuels définitifs pour le logiciel. Les différents éléments graphiques majeurs du logiciel seront mis en avant.

3.5. Rétrospective

Une rétrospective du projet : ses forces, ses faiblesses, etc. Si certaines fonctionnalités n'ont pas pu être implémentées, c'est également ici que les raisons doivent être expliquées.

3.6. Perspectives

Les améliorations du programme : que peut-on améliorer ? que peut-on imaginer en plus ? etc.

4. Code source

- La plateforme doit être développée avec la technologie Javascript / CSS / HTML.
- L'élève ajoutera en annexe de son dossier de conception l'entièreté de son code.
- L'entièreté du code doit être développé par les membres de l'équipe. Chaque fichier doit présenter un commentaire en en-tête avec le nom de la personne ayant codé ce fichier (si cette partie a été codée en duo, les noms des deux, auteurs doivent être indiqués).

5. Système d'évaluation

5.1. Evaluation du travail journalier

Tout au long de l'année, plusieurs remises intermédiaires seront organisées et constitueront l'évaluation du TJ :

1. Dossier de conception : définition du projet (version intermédiaire)
2. Dossier de conception : définition du projet (version finale)
3. Dossier de conception : analyse et conception (version intermédiaire)
4. Dossier de conception : analyse et conception (version finale)
5. Présentation orale en groupe
6. Implémentation du projet (version intermédiaire)
7. Implémentation du projet (version intermédiaire destinée à être présentée à la fête de l'école)

Attention, le TJ ne se limite pas aux remises : l'implémentation du projet pendant les séances seront aussi évaluées, notamment via votre suivi sur Trello.

5.2. Evaluation sommative

La remise finale du projet constitue l'évaluation de l'UAA4.

Le projet est évalué sur base :

- d'une défense orale (30%)
- du dossier de conception (25%)
- du code source (45%)

Évaluation du dossier de conception

Le cahier des charges doit être complet. La mise en page, l'orthographe et la qualité de la rédaction seront prises en compte dans l'évaluation.

Évaluation du programme

Le programme sera évalué selon les points suivants :

- le programme s'exécute correctement au niveau des fonctionnalités attendues,
- le code source est de qualité.

Des bonus seront attribués pour des fonctionnalités supplémentaires non mentionnées dans le cahier des charges.

Toutefois, ces bonus ne seront attribués que si la note du travail de base atteint les 80%.

Évaluation orale

L'évaluation orale consiste à présenter le projet oralement devant les superviseurs. La présentation se déroulera comme suit :

1. Présentation du logiciel
2. Présentation du code source
3. Questions / Réponses sur le logiciel, le code source et le cahier des charges

La présentation du logiciel ainsi que la présentation du code source ne devra pas dépasser 10 minutes.

Dans le cas où le projet est réalisé par deux élèves, ces derniers doivent impérativement démontrer leur participation au projet. À cette fin, le code source devra être documenté de telle manière que l'on puisse identifier clairement l'auteur de chaque ligne de code.

Lors de l'épreuve orale, les questions sur le code source ne porteront que sur le code écrit par l'élève. Par contre, les questions sur le cahier des charges porteront sur l'entièreté de ce dernier.

Attention : le niveau d'exigence sera supérieur pour les projets réalisés à deux, tant au niveau de la rédaction du cahier des charges qu'au niveau de la réalisation du programme.

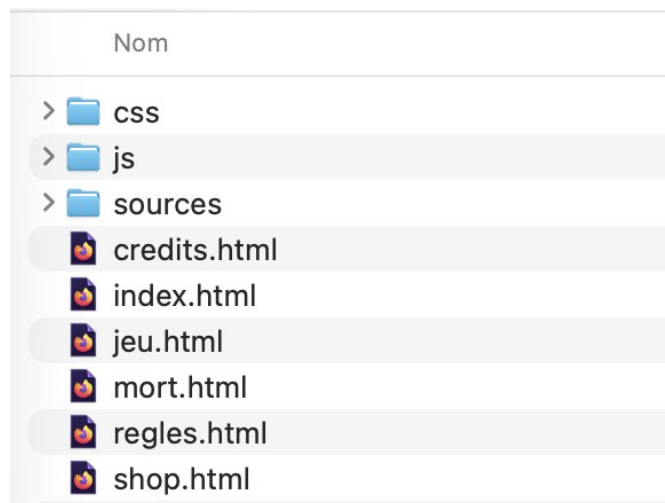
Les différentes grilles d'évaluation sont disponibles à la fin de ce document.

6. Remise finale et nomenclatures

Lors de la remise finale, l'élève remettra une archive compressée nommée 2606_4TT_Nom(s)_TFA.zip comprenant :

- Le dossier de conception et d'analyse du projet nommé 2606_4TT_Nom(s)_DossierConceptionTFA.pdf

- Un dossier nommé 2606_4TT_Nom(s)_CodeSourceTFA contenant le code source du projet. Ce dernier contient les pages HTML (dont l'index qui lancera le jeu) et l'ensemble des sous dossiers suivants :
 - js : dossier contenant les différents fichiers javascript
 - css : dossier contenant les différents fichiers de style
 - lib : dossier contenant éventuellement les librairies / frameworks utilisés pour le projet
 - sources : dossier contenant les autres fichiers nécessaires au jeu (images, vidéos, sons, polices, ...)



Un exemple de code source

7. Contraintes du projet

7.1. Technologies utilisées

Le jeu interactif sera développé sous la technologie Javascript / CSS / HTML. Il devra répondre à la norme W3C, c'est à dire qu'il sera compatible avec TOUS les navigateurs. Par défaut, aucun autre procédé de développement n'est autorisé. Si une autre technologie est souhaitée, cela sera soumis à négociation avec le professeur.

7.2. Organisation du travail en groupe

L'entièreté du code sera développée par les membres de l'équipe. Chaque fichier présentera un commentaire en en-tête avec le nom de la personne ayant codé ce fichier (si cette partie a été codée en duo, les noms des deux auteurs seront indiqués).

7.3. Documentation du code source

Le code du projet sera commenté de manière à permettre à un utilisateur lambda de comprendre et de reprendre facilement le code. C'est-à-dire que pour chaque partie de code répondant à une tâche spécifique, le développeur aura décrit en commentaire ce que le code réalise.

Tout commentaire inutile sera sanctionné dans la cotation.

Exemple de **commentaire inutile** :

```
// Création d'une variable score égale à 0  
let score = 0;
```

Exemple de **commentaire utile** :

```
// Initialisation du score au début de la partie  
let score = 0;
```

7.4. Contraintes fonctionnelles

Le jeu fonctionnera en local et ne nécessitera aucune connexion à internet pour fonctionner.

Le jeu comprendra au minimum un menu principal qui donnera au moins accès à

- une nouvelle partie
- aux règles du jeu
- aux crédits

En tout temps, l'interface présentera un pied de page (footer) fixe reprenant les noms des membres de l'équipe, le nom du projet et l'année académique.

8. Indicateurs de qualité

8.1. La capacité fonctionnelle

« *Le jeu fait-il ce qu'il est censé faire ?* »

La capacité fonctionnelle est la capacité qu'ont les fonctionnalités d'un logiciel à répondre aux exigences et besoins explicites ou implicites des usagers. En font partie la précision, l'interopérabilité, la conformité aux normes et la sécurité.

Exemple : Dans un jeu de Snake , si le serpent mange une pomme, son corps doit s'allonger d'un bloc et le score doit augmenter de 1 point. Si le score ne bouge pas, la capacité fonctionnelle est défailante.

8.2. La facilité d'utilisation

« Comment jouer sans devoir lire un manuel de 50 pages ? »

Elle porte sur l'effort nécessaire pour apprendre à manipuler le logiciel. En font partie la facilité de compréhension, d'apprentissage et d'exploitation et la robustesse. Un autre exemple, une utilisation incorrecte n'entraîne pas de dysfonctionnement.

Exemple : Au lieu d'écrire "Appuyez sur la touche 34 pour sauter", utilisez les flèches du clavier ou les touches ZSQD qui sont naturelles pour un joueur. L'ajout d'un bouton "Rejouer" bien visible après une partie perdue améliore aussi cet indicateur.

8.3. La fiabilité

« Le jeu plante-t-il ou s'arrête-t-il brusquement ? »

C'est-à-dire la capacité d'un logiciel de rendre des résultats corrects quelles que soient les conditions d'exploitation. En font partie la tolérance aux pannes - la capacité d'un logiciel de fonctionner même en étant handicapé par la panne d'un composant (logiciel ou matériel).

Exemple : Si un élève entre son nom dans le menu et qu'il tape des symboles bizarres (comme @#\$%), le jeu ne doit pas s'éteindre ou afficher une page blanche. Il doit rester stable quelles que soient les actions imprévues de l'utilisateur.

8.4. La performance

« Le jeu est-t-il une usine à gaz ? »

C'est-à-dire le rapport entre la quantité de ressources utilisées (moyens matériels, temps, personnel), et la quantité de résultats délivrés. En font partie le temps de réponse, le débit et l'extensibilité - capacité à maintenir la performance même en cas d'utilisation intensive.

Exemple : Si le jeu met 10 secondes à charger une image ou si le personnage se déplace avec des saccades (le « lag »), la performance est mauvaise. Un bon code JavaScript doit permettre au jeu de tourner de façon fluide, même sur un vieil ordinateur de l'école.

8.5. La maintenabilité

« Sera-t-il facile de modifier ou d'améliorer le code plus tard ? »

Elle mesure l'effort nécessaire à corriger ou transformer le logiciel. En font partie l'extensibilité, c'est-à-dire le peu d'effort nécessaire pour y ajouter de nouvelles fonctions.

Exemple : Si vous avez utilisé une variable `vitesse = 5` au début de votre code, vous pouvez changer la difficulté du jeu en modifiant un seul chiffre. Si vous avez écrit « 5 » manuellement à 100 endroits différents dans votre code, votre projet n'est pas maintenable.

8.6. La portabilité

« Le jeu peut-il fonctionner partout ? »

C'est-à-dire l'aptitude d'un logiciel de fonctionner dans un environnement matériel ou logiciel différent de son environnement initial. En font partie la facilité d'installation et de configuration dans le nouvel environnement.

Exemple : votre jeu doit fonctionner de la même manière si vous l'ouvrez avec Google Chrome, Mozilla Firefox ou Microsoft Edge. Il doit aussi être facile à installer : il suffit de copier votre dossier sur une clé USB et de lancer `index.html` pour que cela marche.

GRILLE D'EVALUATION IMPLEMENTATION DU PROJET

Grille d'évaluation					
Critères	Indicateurs	Points	Acquisition		
			A	N-A	E-A
Respect des normes, règles et consignes	A respecté les délais (MALUS) A remis le projet selon une nomenclature correcte (MALUS)				
Maitrise technique et processus	Maitrise des techniques de programmation Le projet ne présente aucun bug L'élève a amené des éléments nouveaux (recherches) Le code est documenté Le code est optimisé et permet sa maintenabilité				
Produit fini	Le projet représente un ensemble fonctionnel. L'élève a implémenté tous les contenus énoncés dans son dossier de conception. Le projet est original, les règles graphiques sont respectées et cohérentes. Le projet est ergonomique et facile à prendre en main.				
Communication	Le projet ne présente aucune faute d'orthographe.				
Pondération		/100			

GRILLE D'EVALUATION DOSSIER DE CONCEPTION

Grille d'évaluation					
Critères	Indicateurs	Points	Acquisition		
			A	N-A	E-A
Respect des normes, règles et consignes	A respecté les délais (MALUS) A remis le fichier selon une nomenclature correcte (MALUS)				
Produit Fini	Le dossier est complet. • Définition du projet • Descriptifs des interfaces et cas d'utilisation • Tâches à réaliser • Aperçus visuels • Rétrospective • Perspectives Le dossier explique les choix de conception de manière complète.				
Communication	Le dossier de conception ne comprend pas de faute d'orthographe. La mise en page, la structure des textes et leur contenu permettent une lecture facile et une compréhension totale du dossier.				
Pondération		/100			

GRILLE D'EVALUATION

DEFENSE ORALE INDIVIDUELLE

Grille d'évaluation					
Critères	Indicateurs	Points	Acquisition		
			A	N-A	E-A
Technique et processus	L'élève prouve le développement d'une part équitable du projet. L'élève peut vulgariser avec aisance le fonctionnement général de son code. L'élève maîtrise et peut restituer une explication claire d'un algorithme particulier. L'élève maîtrise les détails techniques propre aux langages de programmation utilisés. L'élève est capable de proposer une correction de son code et/ou une ou plusieurs fonctionnalité(s) supplémentaire(s).				
Pondération		/100			

FEUILLE DE DÉFINITION DU PROJET

Nom de / des élèves :

.....

.....

.....

Choix du projet (cochez le projet choisi):

- ☐ Le Taquin
- ☐ Un livre dont vous êtes le héros
- ☐ Les dames
- ☐ Snake
- ☐ Breakout
- ☐ Jeu proposé :

Signature de/des élèves :
professeur :

Signature du

Date :